Energy Consumption of the DTSVLIW Architecture

Felipe Thomaz Pedroni · Fernando Líbio L. Almeida · Alberto F. De Souza

Department of Informatics, Federal University of Espírito Santo (UFES), Brazil

5th Workshop on High-Performance Computing (WSCAD 2004) · Foz do Iguaçu, October 27–29, 2004

First author: undergraduate research fellow, PIBIC/UFES/CNPq.

Complete English translation of the original Portuguese conference paper (proceedings pages 82–89). Translation prepared with AI assistance for author review. Equations, experimental figures, values, and references reproduce the source; figure captions and explanatory text are in English.

Abstract

In this work, we present a study of the energy consumption of the DTSVLIW architecture. We implemented a version of our DTSVLIW simulator capable of measuring both dynamic and static energy consumption. We compared these results with those of simulators for the Alpha 21264 processor architecture, namely Wattch and Hotleakage. The experiments showed that the DTSVLIW architecture consumes considerably less energy than the Alpha 21264 processor.

1. Introduction

Until recently, power consumption and heat dissipation were relevant concerns primarily to the designers of portable-computer processors. However, power and heat are expected to become factors limiting the performance of any microprocessor in the future [10].

The maximum possible number of transistors in a processor integrated circuit already exceeds 100 million, and it will soon be possible to implement processors with up to one billion transistors [5]. Nevertheless, switching such a large number of transistors at high frequency results in substantial current consumption and heat generation. Current high-performance processors require forced ventilation or even more elaborate cooling mechanisms. Consequently, demand is growing for high-performance, low-power processors.

In this work, we examine the dynamic and static power consumption of the Dynamically Trace Scheduled Very Long Instruction Word (DTSVLIW) architecture [6, 7], which has not yet been physically implemented. To place the findings in the context of existing processors, we experimentally compare the power consumption of the Alpha 21264 processor [2] against that of a hypothetical DTSVLIW implementation with equivalent hardware. To conduct the experiments, we implemented energy-consumption instrumentation in our DTSVLIW simulator. For the Alpha 21264, we used the publicly available Wattch simulator [4] for dynamic power and Hotleakage [17] for static power. Our DTSVLIW simulator, Wattch, and Hotleakage are all based on SimpleScalar 3.0 (www.simplescalar.com), enabling an appropriate comparison of their results.

2. The DTSVLIW Architecture

Figure 1 shows a block diagram of the DTSVLIW architecture, which has two modes of execution: scalar and VLIW. Whenever a code segment is encountered for the first time, it is executed in scalar mode by the Primary Processor, a simple pipelined processor capable of executing no more than one instruction per cycle; instructions requiring more than one cycle prevent subsequent instructions from proceeding to the execution stage of the Primary Processor pipeline (Figure 1).

In scalar mode, code is fetched from the Instruction Cache by the Primary Processor. As instructions are sent to execution, they are also sent to the Scheduling Unit (the dashed line in Figure 1), which schedules them into VLIW instruction blocks in a Scheduling List. These blocks are saved in the VLIW Cache, with each block address equal to the address of the first instruction in the code segment from which it originated. If the same code segment is executed again, it may be present in the VLIW Cache. The Fetch Unit detects this situation, fetches VLIW instructions from the VLIW Cache, and sends them to the VLIW Machine for execution in VLIW mode. Instructions whose latencies exceed one cycle remain in the VLIW execution stage for more than one cycle; that stage may or may not be fully pipelined, depending on the instruction.

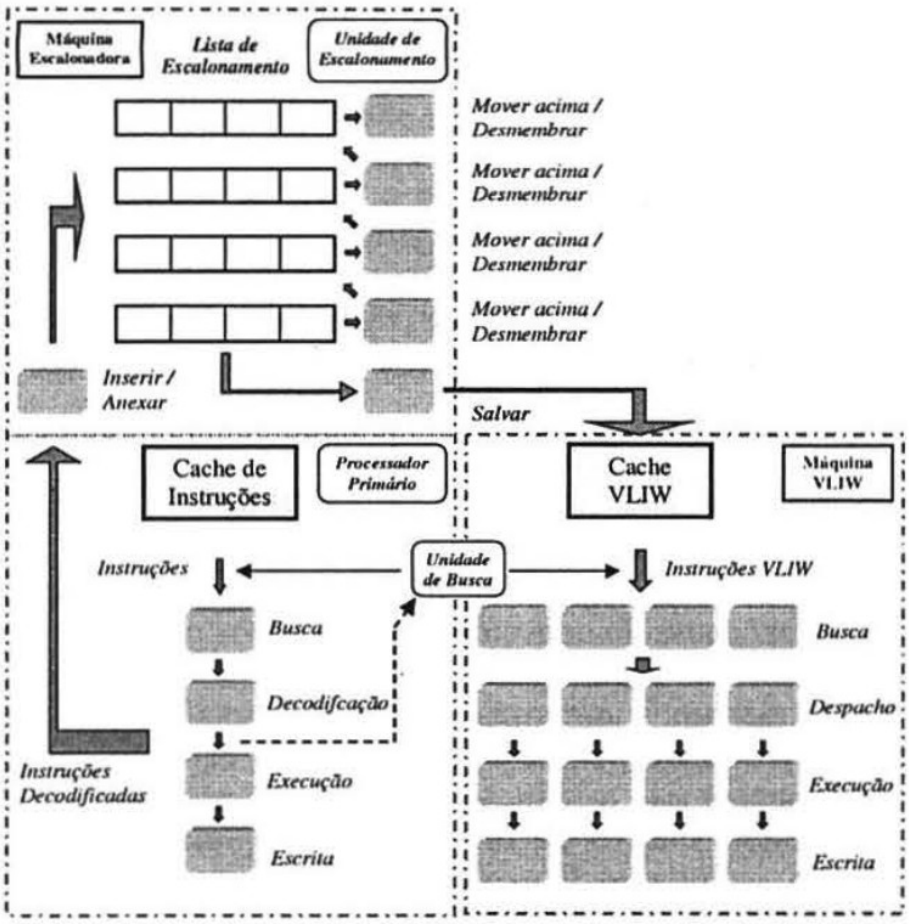


Figure 1. The DTSVLIW architecture (diagram reproduced from the original paper).

Figure terminology (English): Máquina Escalonadora = Scheduling Machine; Lista de Escalonamento = Scheduling List; Unidade de Escalonamento = Scheduling Unit; Processador Primário = Primary Processor; Cache de Instruções = Instruction Cache; Unidade de Busca = Fetch Unit; Máquina VLIW = VLIW Machine; Busca = Fetch; Decodificação = Decode; Execução = Execute; Escrita = Write-back; Salvar = Save.

The DTSVLIW architecture can execute ordinary sequential code in VLIW mode. The amount of parallelism available in its VLIW Machine depends only on the technology used for its implementation. In a DTSVLIW processor, the Scheduling Machine provides object-code compatibility, while the VLIW Machine provides VLIW performance and simplicity.

DTSVLIW is an alternative to superscalar architecture and can be scaled to provide as much or more hardware parallelism than a superscalar processor can exploit. However, as energy consumption increasingly affects architectural choices, DTSVLIW will be selected over superscalar designs only if it also offers power advantages. No quantitative comparison of power consumption between these two architectures had previously been performed. The principal motivation for this study was therefore to investigate DTSVLIW as a potential lower-power alternative to superscalar architecture.

We quantitatively compare the power consumption of the Alpha 21264 superscalar processor with that of a DTSVLIW processor with similar hardware complexity and performance. The resulting evaluation allows us to quantify the benefits of choosing DTSVLIW over a superscalar architecture.

3. Power Consumption

3.1. Operating frequency and voltage

Equation I expresses the relationship between the operating frequency and supply voltage of a CMOS transistor [10]:

$$f \propto (V - Vth)^{\alpha} / V \qquad \text{(Equation I)}$$

In Equation I, V is the supply voltage, Vth is the threshold voltage, and α is an experimentally derived constant, approximately 1.3 for the technologies considered in this paper.

Using industry data, Equation I can be adapted to establish a relationship between voltage and frequency representative of the current technological generation. We define normalized operating voltage Vnorm and normalized frequency fnorm relative to the maximum supply voltage Vmax and maximum operating frequency fmax. These quantities yield the following linear relationship [10]:

$$Vnorm = \beta_1 + \beta_2 \cdot fnorm \qquad \text{(Equation II)}$$

Here $\beta_1 = Vth / Vmax$ and $\beta_2 = 1 - \beta_1$. Equation II is consistent with recent industrial observations. For f = 0, Vnorm = Vth / Vmax, approximately 0.3 for the technologies examined. It also indicates that reducing frequency by a percentage p of fmax permits a smaller percentage reduction in supply voltage. Assuming $\beta_1 = 0.3$, a 50% frequency reduction (fnorm = 0.5) allows voltage to decrease by only 35% (Vnorm = 0.65). Conversely, halving the supply voltage (Vnorm = 0.5) requires a frequency reduction of more than half (fnorm ≈ 0.3).

3.2. Overall power consumption

Equation III expresses the total power consumption of a CMOS circuit as the sum of its dynamic and static components [10]:

$$P = f \cdot A \cdot C \cdot VDD^2 + VDD \cdot Ileak \qquad \text{(Equation III)}$$

The first term models the dynamic power associated with charging and discharging circuit capacitances. The second models static power associated with leakage current Ileak. Equation III ignores short-circuit current, which flows momentarily between supply and ground when a CMOS circuit switches. Because that contribution is relatively small, it can be absorbed into the first term.

3.3. Analysis of dynamic power consumption

A CMOS inverter, illustrated in Figure 2, helps explain the sources of dynamic power consumption. The inverter is the basic component of CMOS digital circuits, the technology used for the great majority of contemporary processors. Once the power consumption of a single inverter is formulated, the same reasoning can be extended to more complex circuits.

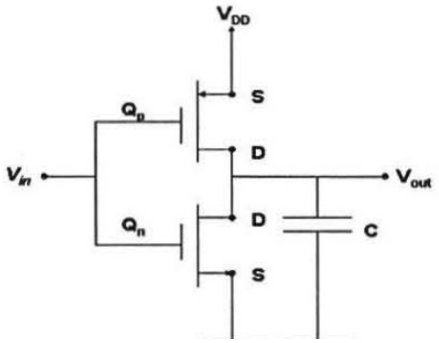


Figure 2. A CMOS inverter (original circuit diagram).

A CMOS inverter uses two enhancement-mode MOSFETs: one n-channel and one p-channel transistor. In Figure 2, VDD is the supply voltage (V in Equation I), and C represents the combined internal capacitances of transistors Qn and Qp, the interconnect capacitance between the inverter output node and the inputs of other gates, and the capacitances of those load gates.

Suppose that immediately before t = 0, Vin = 0, Vout = VDD, and the energy stored in the fully charged capacitor is ½ C VDD^2. At t = 0, Vin becomes VDD: Qp switches off and Qn conducts. Qn discharges C, and by the end of the discharge interval the stored energy ½ C VDD^2 has been removed and dissipated in Qn.

Now consider the other half of the cycle, when Vin returns to zero. Qn switches off and Qp conducts, recharging capacitor C. Let the instantaneous current supplied through Qp be i. That current is drawn from the VDD supply, so the energy provided by the source during charging is

$$\int \mathrm{VDD} \cdot i \cdot dt = \mathrm{VDD} \int i \cdot dt = \mathrm{VDD} \cdot Q \qquad \text{(Equation IV)}$$

where Q is the charge supplied to the capacitor, equal to C VDD. Therefore, energy drawn from the source during charging is C VDD^2. At the end of the charging interval the capacitor voltage is VDD and its stored energy is again ½ C VDD^2. Thus half the energy delivered by the source during charging is dissipated in Qp.

It follows that C VDD^2 is dissipated during each complete switching cycle of the inverter. If the inverter is switched at f A cycles per second, where f is clock frequency and A is an activity factor no greater than one, the dynamic power dissipated is [10]:

$$P_{dynamic} = f \cdot A \cdot C \cdot \mathrm{VDD}^2 \qquad \text{(Equation V)}$$

For the technologies considered in the paper, dynamically dissipated power was the dominant component of processor power consumption. For less aggressively scaled technologies, total power could therefore be approximated by Equation V. The VDD^2 term suggests that lowering supply voltage is an especially effective means of reducing consumption. However, Equation II also shows that halving supply voltage requires a reduction of operating frequency by more than half.

3.4. Analysis of static power consumption

Recent technological advances have enabled transistors with channel widths below 100 nm. Unfortunately, reducing channel width (and thus transistor size) exacerbates static power consumption, leading to the possibility that static power will dominate total processor power. The 2001 International Technology Roadmap for Semiconductors [14], for example, predicted that static power dissipation would overtake dynamic dissipation for fabrication processes below 65 nm.

Static power arises from leakage current through transistors while they are in steady states, rather than switching. Leakage current Ileak results from the combination of subthreshold leakage Isub and gate-oxide leakage Iox [10]:

$$I_{leak} = I_{sub} + I_{ox} \qquad \text{(Equation VI)}$$

Subthreshold leakage current

Subthreshold leakage depends on threshold voltage and supply voltage and is expressed in the source as [10]:

$$I_{sub} = k_1 \cdot W \cdot \exp(-V_{th} / (n\, V\theta)) \cdot (1 - \exp(-V / V\theta)) \qquad \text{(Equation VII)}$$

The constants k_1 and n are determined experimentally; W is transistor channel width; and $V\theta$ is thermal voltage. At room temperature $V\theta$ is approximately 25 mV and increases linearly with temperature. If Isub increases enough to raise temperature, $V\theta$ increases too, which increases Isub further and may produce thermal runaway.

Equation VII suggests two ways to reduce Isub: decrease V, driving the factor in parentheses toward zero, or increase Vth. Equation I, however, shows that increasing Vth reduces operating speed, as does reducing the supply voltage (Section 3.1). Channel width W is another factor contributing to subthreshold leakage; some designers use it when estimating total subthreshold leakage current.

Gate-oxide leakage current

A simplified expression for gate-oxide leakage is [10]:

$$I_{ox} = k_2 \cdot W \cdot (V / T_{ox})^2 \cdot \exp(-\alpha \cdot T_{ox} / V) \qquad \text{(Equation VIII)}$$

Here k_2 and α are experimentally derived, and Tox is the thickness of the oxide dielectric separating the gate from the channel. Increasing Tox reduces gate-oxide leakage, but it also degrades transistor performance because oxide thickness must decrease as fabrication technology scales. Increasing Tox is therefore not a straightforward solution.

4. Related Work

4.1. Power-consumption simulators

Wattch models a superscalar processor and its dynamic power consumption [4]. Its main advantage over physical-level simulation tools is speed: it can run up to one thousand times faster, with reported estimation error in the range of 10–13% [4].

Hotleakage [17] is described as the first architectural simulator capable of estimating statically dissipated power. It computes static consumption but is limited to hardware structures containing large numbers of memory cells, principally caches and register files. Since these structures occupy a substantial portion of chip area, this method provides a useful approximation of total static power.

4.2. Wattch modeling

Wattch uses parameterized power models for the hardware units found in modern superscalar processors. It calculates dynamic power cycle by cycle from Equation V, using parameters for every modeled device. Capacitance C is estimated from transistor sizes and interconnections; VDD and f depend on the selected fabrication technology and the architectural configuration; and activity factor A depends on each unit's design and the program being executed. If an activity factor is not measured for a unit, Wattch assumes a value of 0.5.

The capacitance model in Wattch draws on the models of Wilton and Jouppi [16] and of Palacharla, Jouppi, and Smith [12, 13]. Each hardware unit is divided into stages forming RC circuits. The capacitance of each stage is estimated, and the unit capacitance is obtained by summing those values. The simulator also supports three conditional-clocking styles: mechanisms that deliver a clock to a unit only when it is needed in a given cycle. Contemporary processors make aggressive use of such techniques to deactivate hardware and reduce its consumption when idle.

Hardware structures modeled by Wattch include (1) array structures such as caches, register files, and branch-prediction tables; (2) content-addressable memories (CAMs), including TLBs, issue queues, and reorder buffers; (3) combinational logic and interconnection networks, such as functional units and dependency-checking logic; and (4) the clock distribution network.

4.3. Hotleakage modeling

Hotleakage is a configurable module added to Wattch for calculating static power. It estimates the static power dissipated by individual cells of interest (for example, an SRAM cell), then aggregates these estimates at the architectural level. Its models for individual microarchitectural structures are based on Equations VII and VIII.

At the time of the study, Hotleakage modeled leakage current in caches and register files. Parameters included oxide thickness Tox, supply voltage VDD, structure temperature, transistor threshold voltage for particular structures, inter-die supply-voltage variation, and variations in transistor channel width W.

5. Modeling the Power Consumption of the DTSVLIW Architecture

We implemented a new version of our DTSVLIW simulator capable of measuring the power consumed by the architecture's hardware units. To make the simulator perform the required experiments, we modified its source code. The simulator was written in C and ran under Linux; the added functions estimated the power consumed by the various DTSVLIW hardware units.

We used power models equivalent to those employed in Wattch and Hotleakage. Because all three simulators are based on SimpleScalar, adaptation was facilitated. The implementation considered the organization of the Alpha 21264 superscalar functional units and clock distribution network [3, 8], the power models proposed in [16, 12, 13], the Wattch [4] and Hotleakage [17] implementations, and parameters for the CMOS fabrication technologies considered in [10, 15, 9, 18].

6. Methods

The goal of adding power measurements to the DTSVLIW simulator was to provide a preliminary assessment of the power that a physical implementation of the architecture might consume. To interpret the DTSVLIW results relative to existing processor designs, we compared it with an equivalently configured Alpha 21264. For the Alpha, we combined the two simulators described in Section 4: Wattch [4] and Hotleakage [17].

6.1. Benchmark programs

Both simulators (DTSVLIW and Alpha 21264) accept executables produced by conventional compilers targeting the Alpha instruction set. We used a subset of the SPEC CPU2000 executable collection distributed with SimpleScalar. These binaries were produced on an Alpha 21264 system running Digital UNIX V4.0F and compiled with DEC C V5.9-008 (Rev. 1229), Compaq C++ V6.2-024 (Rev. 1229), or Compaq Fortran V5.3-915 (f77 and f90).

The SPEC CPU2000 inputs were taken from the reduced-input suite developed at the University of Minnesota [11]. Those inputs make it possible for selected benchmarks to execute roughly one billion instructions (about one second on an Alpha system of the period) while capturing meaningful performance characteristics. Every program for which these inputs were available ran either to completion or up to a limit of 2.5 billion instructions. Table 1 identifies the selected benchmarks and instruction counts.

Table 1. Benchmark programs used in the experiments

Integer benchmark	Instructions executed	Floating-point benchmark	Instructions executed
bzip2	1,819,782,161	mesa	1,688,627,786
gcc	2,500,000,000	art	1,660,422,409
gzip	1,583,267,837	mcf	794,460,163
parser	2,500,000,000	equake	1,021,625,144
twolf	972,968,480	ammp	1,247,352,121
vortex	1,154,291,894		
vpr	1,567,083,914		

6.2. Basic simulator configurations

The configurations used in the experiments are listed in Tables 2 through 5. The VLIW Cache is relatively small, only 36 KB (Table 2).

Table 2. DTSVLIW configuration

Component	Configuration
Primary Processor	Four-stage pipeline (fetch, decode, execute, write-back); no branch prediction; taken branches cause a two-cycle pipeline bubble; 16 KB, 2-way set-associative Instruction Cache, one-cycle latency.
VLIW Cache	36 KB, 2-way set-associative, one-cycle latency.
VLIW Cache instruction width	6 bytes.
VLIW Machine pipeline	Three stages: fetch, dispatch, execute.
Scheduling List size	Twice the number of LIs in the block.
Registers used for renaming	Integer: 17; floating point: 13; memory: 6.

Table 3. Alpha 21264 configuration

Component	Configuration
Pipeline	Eight stages: Fetch, Slot, Map, Issue, Register Read, Execute, Write-back, and Retire. Fetch/Slot/Map: four instructions per cycle. Issue/Register Read/Execute/Write-back: six instructions per cycle. Retire: 11 instructions per cycle (as specified in source table).
Functional units	Four integer units and two floating-point units.
Issue queues	20 integer instructions; 15 floating-point instructions.
Rename registers	41 integer; 41 floating point; 32 memory (load and store queues).
Branch predictor	Tournament predictor combining (i) a two-level local predictor (1,024 entries of 10-bit local history), (ii) a path-based global predictor (12-bit history indexing 4,096 2-bit saturating counters), and (iii) a choice predictor (4,096 2-bit saturating counters).

Table 4. Instruction latencies

Instruction	Latency (cycles)
Integer arithmetic	1
Integer multiplication	7
Integer load (cache hit)	3
Floating-point addition and multiplication	4
Floating-point divide / square root (single precision)	12–18
Floating-point divide / square root (double precision)	15–33
Floating-point load (cache hit)	3
Unconditional branch	3

Table 5. Memory hierarchy

Component	Configuration
Instruction Cache (Alpha only)	64 KB, 2-way set associative, one-cycle latency.
Data Cache	64 KB, 2-way set associative, three-cycle latency.
Unified L2 Cache	1 MB, direct-mapped, physically indexed, seven-cycle latency.
Main memory	Unbounded capacity in the model; 66-cycle latency including precharge and 54 cycles for pipelined accesses.

7. Experiments

We used four fabrication technologies to assess dynamic power consumption: 70 nm, 100 nm, 130 nm, and 180 nm. However, because the last physical implementation of the Alpha 21264 used 180 nm technology, only the results for that technology are presented in the dynamic-power comparison.

Almost all energy consumed by a processor becomes heat. Accordingly, our power-consumption estimates are reported as watts (W) dissipated as heat. The experiments were executed on the 64-processor cluster at UFES's High-Performance Computing Laboratory (LCAD).

Figures 3 and 4 plot the modeled power dissipated, in watts, by the Alpha 21264 and DTSVLIW processors configured as shown in Tables 2–5, using a 2 V supply, 1 GHz clock frequency, and 0.55 V threshold voltage. Power for each modeled hardware unit was scaled with its access count: a unit was charged its full modeled power if accessed during a cycle; otherwise, 10% of its maximum power was charged [4].

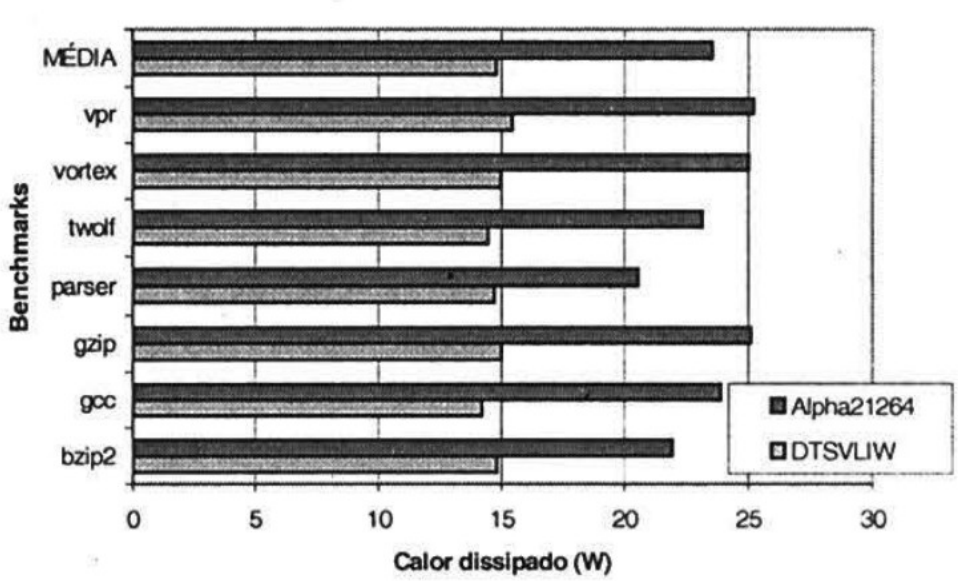


Figure 3. DTSVLIW versus Alpha at 180 nm: integer benchmarks (original data and bars).

Figure terminology (English): Average = mean; benchmark names are unchanged; "Calor dissipado (W)" = power dissipated (W).

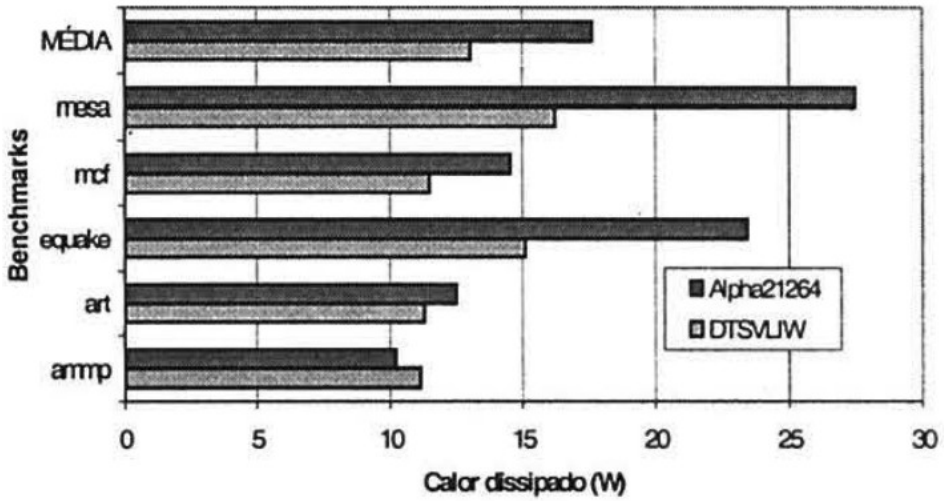


Figure 4. DTSVLIW versus Alpha at 180 nm: floating-point benchmarks (original data and bars).

Figure terminology (English): "Calor dissipado (W)" = power dissipated (W).

As Figures 3 and 4 show, modeled power dissipation is considerably lower for DTSVLIW than for the Alpha 21264 in both integer and floating-point applications. For every integer benchmark, DTSVLIW dissipated less power; average dissipation was 14.80 W, versus 23.54 W for the Alpha 21264—a mean reduction of 37.12%. For floating-point applications, DTSVLIW dissipated substantially less power on mesa and equake, with smaller differences for mcf, art, and ammp. For ammp, the Alpha 21264 performed better in this respect. Across the floating-point set, DTSVLIW averaged 13.06 W against 17.63 W for the Alpha 21264, a mean reduction of 25.92%.

DTSVLIW dissipates less power because it executes in two modes. In scalar mode, the Primary Processor executes code while the Scheduling Unit schedules it and stores it as VLIW instructions. In VLIW mode, the VLIW Machine executes those instructions. DTSVLIW spends most clock cycles in VLIW mode, in which the scheduling unit does not receive clock pulses and its heat dissipation is greatly reduced.

By contrast, the Alpha 21264 superscalar processor schedules instructions in every cycle to exploit instruction-level parallelism. Its scheduling hardware therefore needs clock pulses through nearly all execution. It must also receive functional-unit results to determine which dependent instructions become ready. Three parts of the superscalar scheduling system contribute to the observed dissipation: (i) dispatch logic transferring fetched instructions to reservation stations; (ii) logic forwarding results to reservation stations and enabling ready instructions; and (iii) logic issuing ready instructions from reservation stations to functional units.

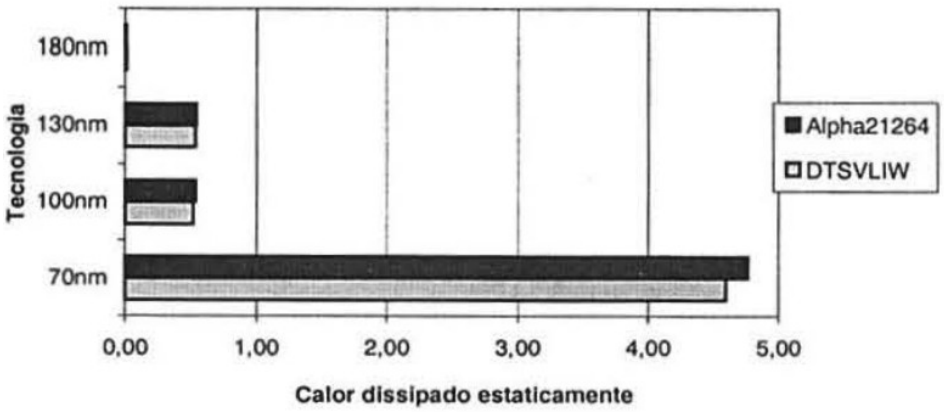


Figure 5. DTSVLIW versus Alpha: modeled static power dissipation at different process nodes.

Figure terminology (English): "Calor dissipado estaticamente" = static power dissipated; "Tecnologia" = fabrication process (nm).

Figure 5 compares the modeled static power dissipated by both processors at the fabrication technologies considered. Static dissipation is very small for both processors at 180 nm but becomes significant at 70 nm. Since the two modeled processors were configured to have nearly equivalent hardware, their static-power dissipation is also very similar. The

paper anticipates that static power may exceed dynamic power in future fabrication technologies [10], while noting ongoing work on approaches for limiting leakage, including high-k dielectrics [10] and carbon-nanotube transistors [1].

8. Conclusion

Our experiments showed that, under the technological assumptions of the Alpha 21264 comparison, the DTSVLIW architecture consumes less power per unit of time (and hence dissipates less heat) than the superscalar architecture. This result suggests that implementing processors based on DTSVLIW would be a competitive approach to the growing problem of processor energy consumption.

Because DTSVLIW has not been physically implemented, its actual power consumption cannot yet be validated or verified. However, because our simulator is based on Wattch, we expect it to exhibit a similar estimation-error range of 10–13% [4]. This is an expectation drawn from the modeling tool, not a measured accuracy claim for fabricated DTSVLIW hardware.

References

[1] P. Avouris, “Supertubes,” IEEE Spectrum, vol. 41, no. 8, pp. 34–39, Aug. 2004.

[2] Digital Equipment Corporation, Alpha Architecture Handbook. Digital Equipment Corporation, 1992.

[3] D. W. Bailey and B. J. Benschneider, “Clocking Design and Analysis for a 600MHz Alpha Microprocessor,” IEEE Journal of Solid-State Circuits, vol. 33, no. 11, pp. 1627–1633, Nov. 1998.

[4] D. Brooks, V. Tiwari, and M. Martonosi, “Wattch: A framework for architectural-level power analysis and optimizations,” Proc. 27th Annual International Symposium on Computer Architecture, pp. 83–94, June 2000.

[5] D. Burger and J. Goodman, “Billion-transistor Architectures,” IEEE Computer, vol. 30, no. 9, pp. 46–49, Sept. 1997.

[6] A. F. De Souza and P. Rounce, “Dynamically Trace Scheduled VLIW Architectures,” Proc. High-Performance Computing and Networking (HPCN’98), LNCS vol. 1401, pp. 993–995, Apr. 1998.

[7] A. F. De Souza and P. Rounce, “Dynamically Scheduling VLIW Instructions,” Journal of Parallel and Distributed Computing, vol. 60, no. 12, pp. 1480–1511, Dec. 2000.

[8] M. K. Gowan, L. L. Biro, and D. B. Jackson, “Power considerations in the design of the Alpha 21264 microprocessor,” Proc. 35th Design Automation Conference, pp. 726–731, 1998.

[9] M. B. Kamble and K. Ghose, “Analytical Energy Dissipation Models for Low Power Caches,” Proc. International Symposium on Low Power Electronics and Design, pp. 343–348, 1997.

[10] N. S. Kim, T. Austin, D. Blaauw, T. Mudge, K. Flautner, J. S. Hu, M. J. Irwin, M. Kandemir, and V. Narayanan, “Leakage Current: Moore’s Law Meets Static Power,” IEEE Computer, vol. 36, no. 12, pp. 68–75, Dec. 2003.

[11] A. J. KleinOsowski and D. J. Lilja, “MinneSPEC: A New SPEC Benchmark Workload for Simulation-Based Computer Architecture Research,” Computer Architecture Letters, vol. 1, June 2002.

[12] S. Palacharla, N. Jouppi, and J. Smith, “Complexity-Effective Superscalar Processors,” Proc. 24th Annual International Symposium on Computer Architecture, 1997.

[13] S. Palacharla, N. Jouppi, and J. Smith, Quantifying the Complexity of Superscalar Processors, University of Wisconsin Computer Science Technical Report 1328, 1997.

[14] Semiconductor Industry Association (SIA), International Technology Roadmap for Semiconductors, http://public.itrs.net.

[15] T. Wada, S. Rajan, and S. A. Przybylski, “An Analytical Access Time Model for On-Chip Cache Memories,” IEEE Journal of Solid-State Circuits, vol. 27, no. 8, pp. 1147–1156, Aug. 1992.

[16] S. Wilton and N. Jouppi, “An Enhanced Access and Cycle Time Model for OnChip Caches,” Technical Report 93/5, DEC Western Research Laboratory, 1994.

[17] Y. Zhang, D. Parikh, K. Sankaranarayanan, K. Skadron, and M. Stan, “Hotleakage: A temperature-aware model of subthreshold and gate leakage for architects,” Technical Report CS-2003-05, University of Virginia, Mar. 2003.

[18] R. Zimmermann and W. Fichtner, “Low-power logic styles: CMOS versus pass-transistor logic,” IEEE Journal of Solid-State Circuits, vol. 32, no. 7, pp. 1079–1090, July 1997.